

Lời giải trận thứ bảy HDU Multi-University 2021

Ban tổ chức

Nguồn gốc: 2021 “MINIEYE Cup” China Collegiate Programming Contest Training, Contest 7 Solutions
contest/hdu-2021-minieye-07-solutions.pdf

1 Fall with Fake Problem

Xét việc nhị phân vị trí của chữ cái đầu tiên trong chuỗi T khác với chuỗi S .

Trong quá trình check, ta muốn dựng một chuỗi có thứ tự từ điển lớn nhất có thể. Khi đó, từ vị trí đang check trở đi, chuỗi T chắc chắn là một dãy giảm. Xét vị trí thật sự đầu tiên khác với chuỗi gốc¹, số vị trí có thể nhiều nhất là 25². Giả sử chữ cái đầu tiên trong T khác với S là c . Khi đó vị trí của mọi chữ cái lớn hơn c đã được xác định³. Ta có thể liệt kê số lần xuất hiện của c ⁴; các vị trí còn lại cần được lấp bằng các chữ cái nhỏ hơn c . Phần này có thể giải bằng ba lô vét cạn, dùng bitset để tăng tốc. Độ phức tạp của check đạt

$$O\left(25\sqrt{k} + 25\sqrt{k}\frac{n}{64}\right).$$

Thực tế quá trình liệt kê c không đầy, vì số lần xuất hiện của các chữ cái lớn hơn c thường không phải là ước của k .

Khi dựng đáp án, phần đầu của chuỗi T đã được xác định bởi các bước nhị phân trước đó; phần sau cũng dùng ba lô tăng tốc bằng bitset, rồi xuất một nghiệm nhỏ nhất theo quá trình DP. Cách làm chuẩn liệt kê chữ cái cụ thể đóng vai trò chữ cái đầu tiên khác nhau, sau đó chạy DP giống trong check và xuất nghiệm theo quá trình DP. Độ phức tạp là

$$O\left(25\sqrt{k}\frac{n}{64}\right).$$

Độ phức tạp cho một bộ kiểm thử là

$$O\left(25\sqrt{k}\frac{n}{64}\log n\right).$$

2 Fall with Soldiers

Trong bản lời giải ngắn, chỉ cần đọc các phần chữ đậm.

Trước hết chú ý tính chất then chốt của đề: **độ dài chuỗi là số lẻ**. Mỗi lần thao tác ta xóa hai ký tự hai bên của mỗi ký tự 1, nên tính chẵn lẻ của độ dài chuỗi không đổi. Dễ thấy nếu có một phương án thao tác làm cho cuối cùng chuỗi còn lại toàn là 1, thì ta có thể tiếp tục xóa chuỗi còn lại cho đến khi chỉ

¹Ví dụ, với zzzzy, khi check(2) thì vị trí đầu tiên khác chuỗi gốc ít nhất là vị trí thứ 4.

²Từ vị trí nhị phân hiện tại, chuỗi T chắc chắn là một dãy giảm; nếu không, ta có thể đưa chữ cái phía sau lên trước để thứ tự từ điển lớn hơn. Vì vậy dãy chỉ có thể giảm nhiều nhất 25 lần, và vị trí đầu tiên khác chuỗi gốc chỉ có thể là lần xuất hiện đầu của mỗi loại chữ cái trong dãy giảm.

³Nếu sau vị trí này vẫn còn chữ cái lớn hơn c , ta có thể chuyển chữ cái đó lên trước để được chuỗi có thứ tự từ điển lớn hơn.

⁴Số lần xuất hiện là một ước của k , nên không nhiều; trong độ phức tạp ký hiệu là $\sqrt{k}$.

còn một ký tự 1. Ngược lại, nếu có phương án xóa chuỗi đến khi chỉ còn một ký tự 1, phương án đó chắc chắn thỏa điều kiện đề bài. Vì vậy điều kiện đề bài cần tìm tương đương với điều kiện có thể xóa chuỗi đến khi chỉ còn một ký tự 1.

Bây giờ xét khi nào một chuỗi có thể bị xóa đến chỉ còn một ký tự 1. Nếu vị trí giữa chuỗi là 1, ta có thể liên tục thao tác trên ký tự 1 này để đạt mục tiêu. Nếu vị trí giữa không phải là 1, ta xét điều kiện để đoạn 0 ở giữa có thể bị xóa. Mỗi ký tự 1 trong chuỗi có số lần thao tác tối đa bằng khoảng cách từ nó đến biên gần nhất; ví dụ 0010000 nhiều nhất chỉ thao tác được hai lần. Một kết luận rất hiển nhiên là ta nên chọn ký tự 1 càng gần tâm càng tốt. Lấy hai ký tự 1 gần tâm nhất ở hai phía trái và phải, rồi xét các đoạn mà thao tác trên từng ký tự có thể xóa. Khi hai đoạn xóa phủ toàn bộ chuỗi, chắc chắn tồn tại một phương án thao tác đưa toàn bộ chuỗi về chỉ còn một ký tự 1⁵. Nếu hai đoạn không phủ hết chuỗi, chắc chắn còn một ký tự 0 nằm giữa hai ký tự 1 và không thể bị xóa bằng bất kỳ cách nào⁶. Điều kiện cuối cùng là: **hai ký tự 1 gần nhất ở hai phía của đường giữa có khoảng cách không vượt quá $(n - 3)/2$** .

Sau khi có điều kiện, ta có thể đếm. Gọi tổng số vị trí cần điền là tot . Chia trường hợp:

- Nếu chính giữa là 1, mọi vị trí còn lại điền tùy ý, đáp án là 2^{tot} .
- Nếu chính giữa không phải là 1, ta chia khoảng theo đường giữa thành hai nửa trái và phải. Gọi $cntl_i$ là số dấu hỏi bên trái vị trí thứ i của nửa trái, $cntr_i$ là số dấu hỏi trong nửa phải có khoảng cách đến vị trí đó không vượt quá $(n - 3)/2$, $locl$ là vị trí ký tự 1 ngoài cùng bên phải trong nửa trái, $locr$ là vị trí ký tự 1 ngoài cùng bên trái trong nửa phải, và $totr$ là tổng số dấu hỏi trong nửa phải. Xét liệt kê vị trí ký tự 1 ngoài cùng bên phải của nửa trái:

- với $i < locl$, không có cách điền dấu hỏi thành 1 nào thỏa điều kiện, nên không có đóng góp;
- với $i \geq locl$, để thỏa điều kiện thì vị trí này phải là 1, mọi vị trí bên phải nó trong nửa trái phải điền 0⁷, và trong nửa phải phải có ít nhất một ký tự 1 thỏa giới hạn khoảng cách. Xét quan hệ giữa $locr$ và vị trí đó, ta nhận được hai dạng đóng góp

$$2^{cntl_i+totr-cntr_i} (2^{cntr_i} - 1) = 2^{cntl_i+totr} - 2^{cntl_i+totr-cntr_i},$$

hoặc

$$2^{cntl_i+totr}.$$

Để thấy ta cần duy trì, với mỗi vị trí dấu hỏi trong nửa trái, hai giá trị 2^{cntl_i+totr} và $2^{cntl_i+totr-cntr_i}$. Đối với đáp án tại $locl$, còn cần duy trì $cntl_i$ và $cntr_i$. Khi thay đổi ký tự ở một vị trí, thông tin trên một đoạn sẽ bị ảnh hưởng bằng các phép nhân/chia 2 hoặc cộng/trừ 1. Ta có thể dùng cây đoạn để duy trì những thông tin này và khi truy vấn thì phân loại như trên để tính đáp án.

3 Fall with Trees

Đặt độ rộng tầng thứ 2 là $w_2 = d$. Khi đó độ rộng tầng thứ k là

$$w_k = \sum_{i=2}^k d \cdot 2^{2-i} = d(2 - 2^{2-k}),$$

⁵Ví dụ 0010010: phạm vi bị xóa bởi hai ký tự 1 trái và phải lần lượt là DD1DD10 và 0010D1D. Hai phạm vi này chồng lên nhau, nên ta có thể hoàn thành mục tiêu. Một cách rõ ràng là liên tục thao tác trên một trong hai ký tự 1 cho đến khi ký tự 1 còn lại nằm ở chính giữa chuỗi, rồi thao tác trên nó. Mỗi lần thao tác trên một ký tự 1 làm vị trí giữa dịch về phía ký tự 1 kia một ô; hai phạm vi xóa phủ nhau nghĩa là ký tự 1 kia luôn có thể đi tới vị trí giữa, nên phương án khả thi.

⁶Ví dụ 0010001: phạm vi xóa của hai ký tự 1 là DD1DD01 và rỗng; có vị trí 00100X1 ở giữa không được ký tự 1 nào phủ. Nếu tồn tại ký tự 1 khác, phạm vi của chúng cũng không thể lớn hơn hai ký tự 1 gần giữa nhất, nên vị trí đó không thể bị xóa.

⁷Trường hợp này gồm cả khi điểm hiện tại là dấu hỏi và khi điểm hiện tại chính là $locl$, nhưng cách tính giống nhau.

và diện tích giữa tầng k và tầng $k + 1$ là

$$S_k = h \frac{w_k + w_{k+1}}{2}.$$

Vì vậy ta cần tính

$$S = \sum_{i=1}^{k-1} S_i = \frac{hd}{2} \sum_{i=1}^{k-1} (4 - 3 \cdot 2^{1-i}).$$

Phần này có thể cộng trực tiếp đến khi đạt độ chính xác yêu cầu, hoặc dùng công thức tổng cấp số nhân.

4 Link with Balls

Gộp hộp có thể lấy $0 \sim k - 1$ quả bóng với hộp chỉ có thể lấy bội số của k quả bóng thành một hộp có thể lấy số bóng tùy ý. Khi đó ta có n hộp có thể lấy tùy ý và một hộp có thể lấy $0 \sim n$ quả bóng. Liệt kê số bóng lấy trong hộp $0 \sim n$; số cách chọn các quả còn lại được tính bằng phương pháp chia thanh. Do đó đáp án là

$$\sum_{i=0}^n \binom{m-i+n-1}{n-1} = \binom{m+n}{n} - \binom{m-1}{n}.$$

5 Link with EQ

Gọi $f(x)$ là kỳ vọng số người sẽ ngồi trong một đoạn trống liên tiếp độ dài x bị chặn ở cả hai đầu⁸. Khi đó:

$$f(x) = f\left(\left\lfloor \frac{x-1}{2} \right\rfloor\right) + f\left((x-1) - \left\lfloor \frac{x-1}{2} \right\rfloor\right) + 1,$$

$$f(1) = f(2) = 0.$$

Tương tự, đặt $g(x)$ là kỳ vọng số người sẽ ngồi trong một đoạn trống liên tiếp nửa mở độ dài x ⁹. Khi đó

$$g(x) = \begin{cases} 0, & x \leq 1, \\ f(x-1) + 1, & x \geq 2. \end{cases}$$

Cuối cùng, gọi $h(x)$ là kỳ vọng số người sẽ ngồi ở một bàn độ dài x . Ta có

$$h(x) = \frac{1}{x} \sum_{i=1}^x (g(i-1) + g(x-i) + 1),$$

nên

$$h(x) = 1 + \frac{2}{x} \sum_{i=1}^{x-1} g(i).$$

Vì vậy chỉ cần tiền xử lý lần lượt $f(x)$, $g(x)$, tổng tiền tố của $g(x)$ và $h(x)$; khi truy vấn thì xuất trực tiếp đáp án.

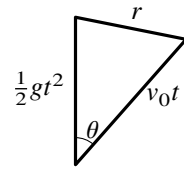
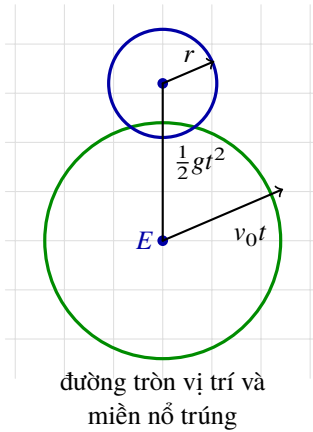
⁸Bị chặn ở cả hai đầu nghĩa là bên trái vị trí trống ngoài cùng bên trái và bên phải vị trí trống ngoài cùng bên phải đều đã có người.

⁹Nửa mở nghĩa là một phía là đầu bàn, phía còn lại đã có người.

6 Link with Grenade

Chọn ngẫu nhiên điểm trên một mặt cầu và tích phân hai lần theo góc về bản chất là giống nhau, vì hai cách này có cùng mật độ diện tích trên đơn vị mặt cầu. Hướng của lựu đạn trên mặt phẳng ngang không ảnh hưởng đến việc nó có nổ trúng người hay không, nên ta chỉ cần xét mặt phẳng thẳng đứng.

Xét đổi hệ quy chiếu: cộng cho cả người và lựu đạn một gia tốc trọng trường ngược chiều. Khi đó lựu đạn chuyển động thẳng đều, còn người chuyển động thẳng biến đổi đều. Sau t giây, vị trí của người là cố định, còn vị trí của lựu đạn là một đường tròn; miễn mà lựu đạn có thể nổ trúng người cũng là một đường tròn. Dựng tam giác tương ứng như hình dưới đây, rồi dùng định lý cos để tìm góc nâng θ mà lựu đạn có thể nổ trúng người.



Theo công thức diện tích chòm cầu, diện tích phần có thể nổ trúng người là

$$2\pi(v_0 t)^2(1 - \cos \theta),$$

còn diện tích mặt cầu là $4\pi(v_0 t)^2$. Vì vậy đáp án là

$$\frac{1 - \cos \theta}{2}.$$

Thay biểu thức $\cos \theta$ nhận được từ định lý cos vào là xong; chú ý xử lý riêng hai trường hợp không thể nổ trúng và chắc chắn nổ trúng.

7 Link with Limit

Dựng đồ thị, nối cạnh từ đỉnh x đến đỉnh $f(x)$. Khi đó quá trình lặp hàm f về bản chất là quá trình đi trên đồ thị. Bài gốc thực ra hỏi liệu trung bình trọng số đỉnh của mọi chu trình có giống nhau hay không. Chỉ cần dùng bất kỳ phương pháp nào để tìm tất cả chu trình và tính trung bình của chúng.

8 Smzzl with Greedy Snake

Trước hết, số lượng lệnh f hiển nhiên là khoảng cách Manhattan giữa hai điểm kề nhau; ta chỉ cần giảm số lần rẽ. Đề bài nói hai điểm kề nhau không nằm trên cùng hàng hay cùng cột, nghĩa là ta cần đi theo hai hướng kề nhau. Nếu hướng hiện tại đã là một hướng cần đi, cứ đi thẳng. Nếu không, chỉ cần xoay một lần là đến được một hướng cần đi, sau đó mô phỏng là được.

9 Smzzl with Safe Zone

Đề bài yêu cầu tìm một điểm trên bao lồi ngoài sao cho khoảng cách gần nhất từ nó đến bao lồi trong là xa nhất. Có thể chứng minh rằng chọn điểm trên bao lồi ngoài tại một đỉnh không bao giờ tệ hơn.

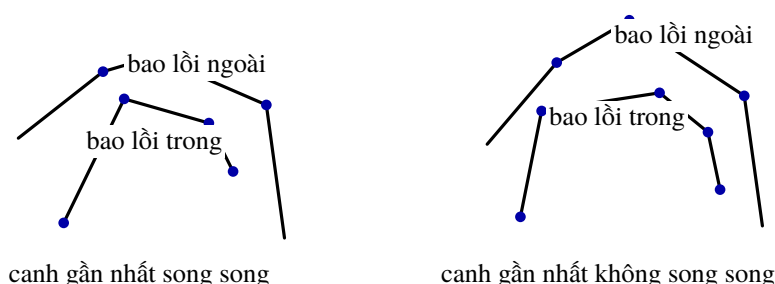
Do đó xét liệt kê từng điểm trên bao lồi ngoài, rồi vét cạn từng cạnh của bao lồi trong để tính khoảng cách nhỏ nhất và lấy giá trị lớn nhất. Cách này có độ phức tạp $O(n^2)$ cho một bộ dữ liệu.

Ta có thể dùng quay thước kẹp để tối ưu quá trình trên. Khi liệt kê các điểm trên bao lồi ngoài theo chiều ngược kim đồng hồ, điểm gần nhất tương ứng trên bao lồi trong cũng quay đơn điệu theo chiều ngược kim đồng hồ. Vì vậy tương tự quay thước kẹp, khi liên tục xét điểm kế tiếp trên bao lồi ngoài, ta liên tục duy trì cạnh gần nhất tương ứng trên bao lồi trong. Cách này có độ phức tạp $O(n)$ cho một bộ dữ liệu.

Chứng minh rằng “chọn điểm trên bao lồi ngoài tại một đỉnh không bao giờ tệ hơn” như sau. Nếu điểm được chọn nằm trên một cạnh của bao lồi ngoài:

1. Nếu cạnh gần nhất của bao lồi trong song song với cạnh này, dời điểm đã chọn trên bao lồi ngoài đến một đầu mút sẽ không tệ hơn.
2. Nếu cạnh gần nhất của bao lồi trong không song song với cạnh này, dời điểm đã chọn trên bao lồi ngoài đến một đầu mút sẽ tốt hơn.

Suy ra chọn điểm tại một đỉnh của bao lồi ngoài không bao giờ tệ hơn.



10 Smzzl with Tropical Taste

Thể tích trà chanh trong bể thỏa phương trình vi phân

$$\begin{cases} V'(t) = (q - p)V(t), \\ V(0) = 1. \end{cases}$$

Giải ra

$$V(t) = e^{(q-p)t}.$$

Thực chất ta cần xét tích phân suy rộng

$$\int_0^{+\infty} V(t) dt$$

có hội tụ hay không. Nguyên hàm là

$$\int V(t) dt = \begin{cases} \frac{e^{(q-p)t}}{q-p}, & q \neq p, \\ t, & q = p. \end{cases}$$

Vì vậy khi $q < p$ thì tích phân hội tụ; khi $q \geq p$ thì phân kỳ.

11 Yiwen with Formula

Gọi cnt_x là số dãy con có tổng bằng x . Khi đó đáp án là

$$\prod x^{cnt_x}.$$

Xét cách tính cnt_x . Một cách đơn giản là đặt $f[i][j]$ là số cách chọn từ i số đầu để có tổng j , rồi làm DP với độ phức tạp $O(n^2)$. Để tối ưu DP này, viết mỗi số a_i thành đa thức $1 + x^{a_i}$, sau đó dùng chia để trị kết hợp NTT. Chú ý mô-đun của đa thức thực ra là

$$\varphi(998244353) = 998244352,$$

nên cần NTT modulo tùy ý. NTT ba mô-đun hoặc FFT tách bit không tối ưu có thể bị quá thời gian vì hằng số quá lớn; bản mã đầu tiên của người ra đề thậm chí chạy chậm hơn vét cạn. Có thể tham khảo phương pháp tối ưu do Mao Xiao nêu trong luận văn đội tuyển quốc gia năm 2016.

Độ phức tạp là $O(n \log^2 n)$.

12 Yiwen with Sqc

Để thấy các chữ cái khác nhau không ảnh hưởng lẫn nhau, nên xét riêng từng chữ cái. Không mất tính tổng quát, xét chữ cái a . Giả sử a xuất hiện cnt lần trong chuỗi gốc. Gọi vị trí xuất hiện thứ i của chữ cái a trong chuỗi gốc là a_i , với $a_0 = 0$ và $a_{cnt+1} = n + 1$. Đặt

$$len_i = a_{i+1} - a_i,$$

tức khoảng cách giữa hai chữ cái a kề nhau.

Trước hết cố định đầu trái tại 1 và di chuyển đầu phải. Khi đó đáp án là

$$\sum_{k=1}^{cnt} len_k k^2,$$

vì khi đầu phải di chuyển giữa hai vị trí a_i kề nhau, số lượng chữ cái không đổi; biểu thức trên chính là tổng theo từng đoạn, mỗi đoạn lấy độ dài nhân với bình phương số chữ cái từ đầu đoạn đến đầu chuỗi.

Liên tục dịch đầu trái và gọi vị trí hiện tại là pos . Khi $pos \leq a_1$, đáp án đều bằng

$$\sum_{k=1}^{cnt} len_k k^2,$$

nên đóng góp của đoạn này là

$$len_0 \sum_{k=1}^{cnt} len_k k^2.$$

Tương tự, khi $pos \in (a_1, a_2]$, đáp án là

$$\sum_{k=2}^{cnt} len_k (k-1)^2.$$

Có thể thấy với $pos_i \in (a_i, a_{i+1}]$ ta có

$$\sum_{k=i+1}^{cnt} len_k (k-i)^2.$$

Đặt

$$ans_i = \sum_{k=i+1}^{cnt} len_k (k - i)^2.$$

Nếu không muốn đọc suy luận chặt chẽ, có thể tự lấy vài ví dụ rồi lấy sai phân hai lần của ans để nhìn ra cách làm.

Gọi dif_i là sai phân của hai giá trị ans kề nhau. Khi đó

$$\begin{aligned} dif_i &= ans_{i+1} - ans_i \\ &= \sum_{k=i+2}^{cnt} len_k (k - i - 1)^2 - \sum_{k=i+1}^{cnt} len_k (k - i)^2 \\ &= \sum_{k=i+2}^{cnt} len_k (k - i - 1)^2 - \sum_{k=i+2}^{cnt} len_k (k - i)^2 - len_{i+1} \\ &= - \sum_{k=i+2}^{cnt} len_k (2k - 2i - 1) - len_{i+1}. \end{aligned}$$

Gọi $diff_i$ là sai phân của hai giá trị dif kề nhau. Khi đó

$$\begin{aligned} diff_i &= dif_{i+1} - dif_i \\ &= - \sum_{k=i+3}^{cnt} len_k (2k - 2(i+1) - 1) - len_{i+2} + \sum_{k=i+2}^{cnt} len_k (2k - 2i - 1) + len_{i+1} \\ &= - \sum_{k=i+3}^{cnt} len_k (2k - 2(i+1) - 1) + \sum_{k=i+3}^{cnt} len_k (2k - 2i - 1) + 3len_{i+2} + len_{i+1} - len_{i+2} \\ &= \sum_{k=i+3}^{cnt} 2len_k + 3len_{i+2} + len_{i+1} - len_{i+2}. \end{aligned}$$

Từ công thức trên có thể dễ dàng duy trì giá trị $diff$, rồi duy trì dif , và cuối cùng duy trì ans .

Cách làm trên là $O(n)$; cách $O(26n)$ có thể bị kẹt hằng số.